International Journal of Information Security (2026) 25:143
https://doi.org/10.1007/s10207-026-01316-z

REGULAR CONTRIBUTION

Design methodology of clock signal manipulation detectors in FPGAs and ASICs to strengthen vulnerable cryptosystems against fault injection

Francisco Eugenio Potestad-Ordóñez[1,3] · Erica Tena-Sánchez[1,3] · José Miguel Mora-Gutiérrez[3] · Antonio José Acosta-Jiménez[2,3]

Received: 17 September 2025 / Accepted: 30 July 2026 / Published online: 18 August 2026
© The Author(s) 2026

Abstract
The growth of the secure Internet of Things has led, among other aspects, to the necessity of ensuring the confidentiality of sensitive user data while simultaneously adhering to stringent constraints regarding area and resource consumption. Attacks on physical implementations of cryptographic primitives are numerous and increasingly sophisticated, making the development of secure crypto-systems a challenge. This paper presents a methodology for designing a lightweight detection scheme for detecting fault injections by clock manipulation in hardware-oriented information encryption algorithms in both FPGAs and ASICs. To demonstrate its versatility and fault coverage, several experimental tests have been performed on an Artix-7 100T FPGA and a TSMC 90nm ASIC that incorporate the proposed schemes on both the standard AES block cipher and the standard Trivium stream cipher. Furthermore, its use against electromagnetic attacks has been evaluated. The experimental results on both technologies show that the scheme is able to detect 100% of both the clock signal manipulation and electromagnetic attacks, with a negligible penalty in area consumption and without frequency degradation.

Keywords Fault attack · Countermeasure · AES · Trivium · Fault injection · IoT · Clock manipulation · Electromagnetic attacks

1 Introduction

The security of communications and, consequently, the protection of user-sensitive data has become a social, scientific, and technological challenge. The Internet of Things (IoT) market is constantly developing and is expected to reach around 40 billion IoT devices by 2030 [1]. In [2], the authors describe the importance of analyzing security in the IoT, considering recent research work on the different stages of the IoT security solution. The authors in [3] and [4] describe the importance of lightweight cryptography as a solution to the problem of securing resource-constrained devices in the IoT. In [5], the authors discuss the risks that exist if attacks on embedded applications used in this area are not considered. Due to this, the development of new cryptographic algorithms that try to protect the information and meet the strong constraints imposed by the applications is constant.

Attacking encryption algorithms by brute force has been shown to be unfeasible because they are mathematically secure with current computation capabilities. However, attacks that are proving to be very effective are those that focus on the physical implementation of these algorithms. These are called Hardware Attacks (HA) and can be divided into two main groups, passive and active attacks. The so-called active attacks can be classified as invasive or non-invasive ones. Invasive attacks are those in which, after the attack, the hardware has been modified. Non-invasive

✉ Francisco Eugenio Potestad-Ordóñez
fpotestad@us.es

Erica Tena-Sánchez
etena@us.es

José Miguel Mora-Gutiérrez
jmiguel@imse-cnm.csic.es

Antonio José Acosta-Jiménez
acojim@us.es

[1] Departamento de Tecnología Electrónica, Escuela Politécnica Superior, Universidad de Sevilla , 41011 Seville, España

[2] Departamento de Electrónicay Electromagnetismo, Facultad de Física, Universidad de Sevilla, 41012 Seville, España

[3] Instituto de Microelectrónica de Sevilla, IMSE, CNM (Universidad de Sevilla, CSIC), 41092 Seville, España

attacks, on the other hand, did not leave proofs of the attack on the target [6, 7].

Within the active non-invasive attacks, the attacker deliberately modifies the operation of the circuit to obtain incorrect operation. These attacks belong to the category of Fault Injection Attacks (FIA) [8], and can be executed by manipulating clock or control signals, power supply, or electromagnetic pulses, among others. Laser beam attacks also belong to the FIA but these are invasive as the circuit needs to be stripped, and a trace of the attack is left behind.

By comparing the correct operation with the faulty operation caused by the fault, the attacker can obtain a part of the secret information contained in the circuit. Within these attacks, the mathematical analysis used to perform cryptanalysis is known as Differential Fault Analysis (DFA). This technique was introduced by Biham in 1997 in [9], where a differential failure analysis of the Data Encryption Standard (DES) cipher was presented. Thanks to this type of mathematical analysis, the attacker can exploit the information generated by the injection of transient faults into the circuit repeatedly. It should be noted that fault attacks are not only exploitable through DFA [9, 10], but there are a multitude of techniques that exploit this type of attack and vulnerabilities such as Ineffective Fault Analysis (IFA) [11], Mutant Differential Fault Analysis (MDFA) [12] and Algebraic Fault Analysis (AFA) [13]. For simplicity in the text, DFA has been considered, as it is one of the most effective fault analysis applied to cryptocircuits.

The security breach in the protection of user information and, therefore, prevention or detection of these types of attack has become a challenge. Therefore, in this paper, we will focus on active non-invasive attacks, of which, specifically on those that manipulate clock signals or control signals to introduce faults into the system and on electromagnetic fault injections (EM).

To face FIA, numerous countermeasures have been proposed for the main NIST encryption algorithms [14], being the most common hardware reduction [15, 16], information redundancy [15, 17–19], temporal redundancy [15], and a combination of them [15, 20]. Our approach is oriented toward the detection of malicious manipulation by an attacker, being applicable to all systems vulnerable to FIA, as, for instance, the main components of cryptographic hardware Root of Trust (RoT), such as Physical Unclonable Functions (PUFs), True Random Number Generators (TRNGs), or symmetric ciphers. To evaluate the proposed design methodology, symmetric ciphers have been chosen as a demonstrator vehicle in this paper.

The main contributions of this work are listed below.

- The presentation and evaluation of a clock signal manipulation detection methodology oriented to both FPGA and ASIC technologies. In the case of FPGAs, Xilinx Artix-7 commercial boards have been used. In the case of ASICs, a 90-nm TSMC integrated circuit, has been designed, shipped to be manufactured, and implemented on a board specifically designed for its use.
- The design methodology and the proposed schemes have been incorporated in an Advanced Encryption Standard (AES) block cipher and in a Trivium stream cipher. The vulnerability of both algorithms to clock manipulation attacks has been studied and protection schemes have been designed following the proposed methodology, being assessed through simulations and experimentally.
- The proposed schemes have been experimentally tested using commercial fault injection platforms such as NewAE ChipWhisperer Husky for the faulty clock generation and ChipShouter for the electromagnetic fault injection. To this end, as few assumptions as possible have been made to bring the attack model as close to reality as possible.
- The analysis of the requirements to be met by clock manipulation attacks to achieve effective attacks and the influence of EM pulses on the internal signals of the circuits are also presented.
- Finally, a comparative analysis of the most pertinent protection schemes is provided, focusing on the balance between hardware resources, frequency degradation, and the level of protection offered.

The paper is organized as follows. Section 2 describes the previous work, focusing on the attack methodology, countermeasures, delay-based countermeasures, and cryptocircuits used as a test vehicle. Section 3 presents a description of the proposed solution. Section 4 presents the design methodology of the detectors. Section 5 presents the experimental results. Finally, Sect. 6 presents the conclusions derived from the work.

2 Previous work

Symmetric ciphers in general, block and stream ciphers in particular, have been shown to be vulnerable to FIA and DFA attacks [10, 21–24]. The description of the attack methodology against which the proposal of this paper is effective, the countermeasures proposed in the literature, and the description of the test vehicles used as proofs of concept are described below.

2.1 Fault injection methodology

Clock signal manipulation is a widely known and used technique for injecting faults into circuits since it was introduced in [8]. Specifically, it is one of the main techniques used for cryptographic circuits as a means to obtain malfunctions

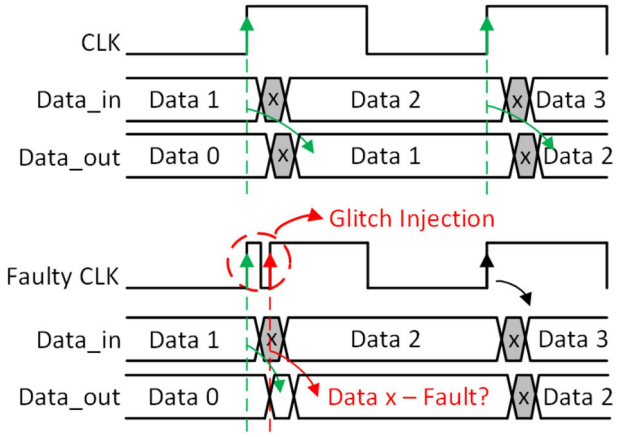


Fig. 1 Representation of transient fault injection using clock manipulation

and exploit vulnerabilities in the physical implementations of these circuits. Figure 1 shows a basic representation of how this technique works. Supposing the data are sampled with the rising edge of the clock *clk*, since the data needs a minimum time to be stable, if a new active clock edge occurs when the data are not stable (clock glitch), violations of setup and hold times in the flip-flops of the system can occur, introducing data faults. Considering that, the minimum clock period necessary for a correct circuit work is given by (1), where t_{su} is the setup time, t_h is the hold time, and t_p is the propagation time of the flip-flop. As an example, in Fig. 2 it can be seen the short pulse injection in the main clock signal.

$$T_{clk}(min) = t_{su} + t_h + t_p. \tag{1}$$

Then, if a shorter clock period is used in a certain clock cycle, it is possible to inject transient faults into the cipher, as depicted in Fig. 1. It should be noted that this type of fault injection is typically introduced on the critical paths, namely, the paths or register that are connected to a significant number of logic operations or those with highest delay.

2.2 Countermeasures

To prevent FIA and, in particular, the manipulation of clock signals, and to protect cryptocircuits, several countermeasures have been proposed in the literature [15–20, 25–28]. These countermeasures are based on information redundancy, time redundancy, hardware redundancy, or a combination of them. While some approaches achieve a high level of protection, they do so at the expense of a significant degradation in operating frequency, or, on the contrary, the protection level is not adequate for a certain group of applications, allowing third parties to obtain the secret data. This is why it is necessary to develop solutions where the area

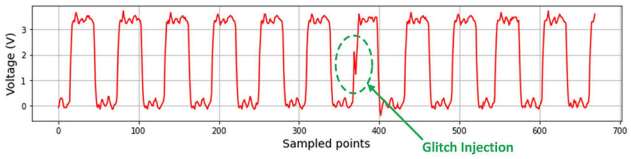


Fig. 2 Short pulse injection in the main clock signal

and frequency penalties are very reduced, allowing one to use them in very constrained applications.

2.3 Delay based countermeasures

The countermeasures listed above base their operation on the detection of faults introduced in cryptographic circuits, such as the change of state of their internal data, not on the detection of manipulation of the signals that control the circuit. However, the use of time delays for the detection of clock signal manipulation is a very useful method to determine if the circuit is under attack and to detect possible fault injections. Within countermeasures based on time delays its main drawback is the design methodology itself, which, depending on the resources needed to implement the protection, may only be applicable to FPGA technologies and hardly applicable to ASICs for final applications closer to real scenarios. In the work presented by Deshpande et al. [25] they use a ring oscillator for the detection of the manipulation of the clock signal when its frequency continuously increases or decreases over a long period of time. A comparison of the clock signal generated by the ring oscillator with the system clock using two counters is made. The result is stored in non-volatile memory and compared later. If the main clock signal is modified, the comparison will be erroneous and, therefore, it is possible to detect the manipulation. In [26], the authors present a methodology for detecting errors in the FPGA-oriented clock signal, estimating possible glitches that can occur in digital circuits and the determination of their position. This system, on the one hand, makes use of Digital Clock Manager (DCM) specific to Xilinx FPGAs to control the clock phase shift and, on the other hand, uses BRAMs to store the clock calibration and measurements of the capture circuit to compare the result and send it to the PC for processing. In [27], a delay-based detector is implemented for the DES cipher. In this approach, the authors design the delay elements according to the clock period and timing slack of the specific circuit under protection. The design consists of a delay chain connected to two XOR gates, along with two counters that track the number of rising and falling clock edges. Subsequently, two comparisons are performed using dedicated comparators: if the counter value equals zero, a clock signal manipulation is detected. The experimental evaluation on the DES cipher is conducted at three different operating frequencies, requiring a frequency-specific design

for each tested configuration. In [28], the authors propose a detector based on a ROM structure, where different delay values, previously obtained through static timing analysis, are stored. The scheme operates in two distinct phases: an initial design-time phase, in which the delay values are computed and programmed into memory, and a subsequent operational phase. Once the system enters the operational phase, the memory contents remain immutable. The detection mechanism relies on an enable signal that determines whether the correct data is propagated or, in the event of a detected fault, replaced by data generated from a random number generator. A key limitation of this approach is that it fails to operate correctly if the combinational logic delay exceeds the activation time of the enable signal. The detector is based on delay elements and multiplexers, where the selected delay path is determined by the values stored in the ROM. Experimental validation is performed on a SASEBO-G platform using an FPGA implementation, while the reported hardware overhead is estimated for an ASIC implementation without actual fabrication.

2.4 Cryptocircuits used as test vehicle

In this paper, two widely used standard symmetric ciphers have been used as test vehicles. The Advanced Encryption Standard (AES) block cipher [29] and the Trivium stream cipher [30]. Using these two standard ciphers, it is possible to determine the versatility and efficiency of the proposed solution extendable to other ciphers widely used in IoT applications.

- The AES cipher is the NIST standard using the Rijndael algorithm with 128 bit input blocks. Depending on the key size (128, 192, or 256 bits), AES performs the input transformation over multiple rounds (10, 12, or 14, respectively). The round process consists of processing the (16 byte) input state S through the operations SubBytes(), ShiftRows(), MixColums(), and AddRoundKey(). For our tests, we have considered the 128 bit key configuration. This cipher is used in most applications.
- The Trivium stream cipher is one of the finalists in the eSTREAM project and was adopted as the ISO / IEC 29192-3 standard [31]. It is a synchronous stream cipher designed to generate up to 2^{64} bits of key stream from an 80-bit secret key (K) and an 80-bit initialization vector (IV). The cipher architecture is based on three shift registers comprising 288 bits in total, as well as combinational logic to provide feedback. This stream cipher has shown to be a great candidate for IoT applications where the resource and time constraints are very high.

In the case of the AES block cipher, the vulnerability against clock signal FIA could be focused on the SubByte operation and state matrix, and in the case of the Trivium stream cipher it could be focused on the feedback flip-flops of the inner shift register [10, 21–24].

3 Proposed detection scheme

To detect malicious active non-invasive attacks based on manipulations of the clock or control signals, we have designed a system based on time delays and signal sampling. For the sake of simplicity, the clock signal has been selected as a fault injection point in the design. We consider as clock manipulations the injection of narrow pulses into the clock line with the aim of violating the setup and hold times in flip-flops. This technique is a powerful attack method, widely known and used, and it is possible to implement it using cheap and commercial tools. It is important to note that the proposed detection system does not focus on preventing or countering the introduction of faults, but rather on detecting whether the system is under attack, with the aim of being as adaptable as possible to any system that needs to know if it is under attack and, depending on its needs, designing an appropriate response.

The presented approach is based on the use of a configurable series of time delays and the inclusion of standard D flip-flops in the main clock line of the circuit, as it is shown in Fig. 3. At the data input (D) of the flip-flop, the clock signal is applied, while the clock input of the flip-flop is driven by a delayed version of the same clock signal. This configuration ensures that the data input is sampled using a delayed clock edge. In Fig. 3, the AND gate is used to trigger the error signal whenever at least one detection module identifies that the circuit is under attack. If no modification is produced in the clock line, the scheme always samples the high level of the clock signal, but, on the contrary, if a modification is produced in the main clock, the scheme can sample a low level of the clock, as depicted in Fig. 4. As a result of clock manipulation, the error signal is set to a low value for a short time as an alarm, allowing one to detect the attacks. It is important to note that an AND gate is used in this example to show the integration of multiple detectors operating simultaneously. The use of more than one detector is intended to address a broader range of potential fault frequencies; however, a single detector may be sufficient for the protection of the circuit.

Taking into account that each circuit and technology that implement the crypto algorithm can be different and the length of the short pulse that can produce the fault injection could be different for each crypto circuit, the detection scheme must be designed to cover different length of clock short pulses. For this, depending on the added delay to the

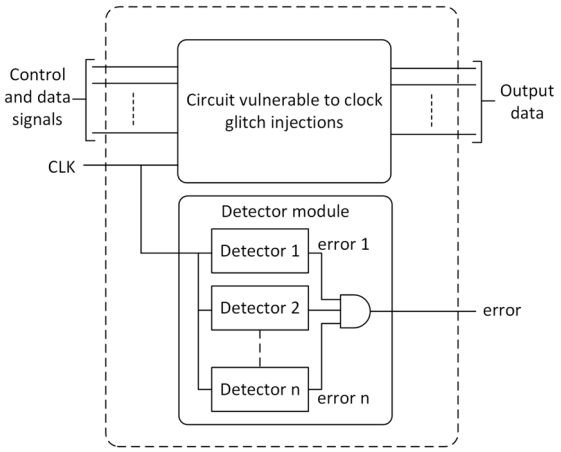


Fig. 3 Representative application of the detection scheme to the circuit

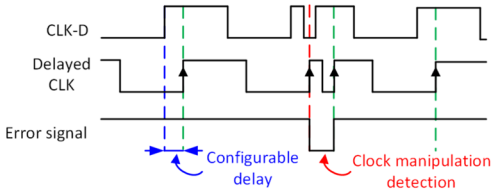


Fig. 4 Descriptive waveform on the operation of the scheme

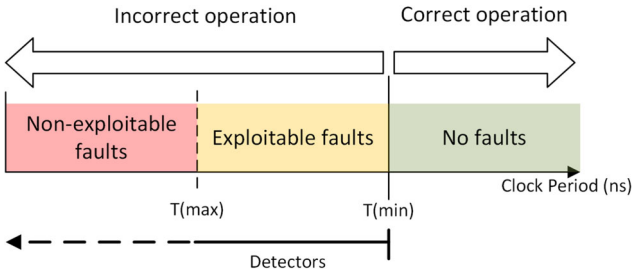


Fig. 5 Representation of the range of periods to be considered

clock signal, different types of short pulse can be detected. If equation (1) is considered and we take into account that the minimum period for the crypto circuit is $T_{clk}(min) = (F_{clk}(max))^{-1}$ and the period that can cause fault injections, namely the faulty period, is $T_{fault} < T_{clk}(min)$, then we can define (2), where d is the total delay used for the clock line and d_{cell} is the delay value of the delay cell type used for the configuration. By adding different delays in a string, it is possible to delay the clock signal until the desired clock range. The group of delays added to the clock line must be greater than half of the faulty period and less than the total faulty period.

$$d = \sum_{i=0}^{n} d_{cell} i \Longrightarrow d \in [0.5T_{fault}, 0.99T_{fault}]. \tag{2}$$

In the case of ASIC, depending on the integration technology, it is possible to use different cells with specific delay for each one. For FPGAs, if different FPGA families are considered, inverter cells have a different propagation time. This aspect allows to perform very specific clock detectors using (3), where each dx_{cell} represents different types of delay cell for the desired technology.

$$d = \sum_{i=0}^{n_1} d1_{cell} i + \sum_{i=0}^{n_2} d2_{cell} i + \sum_{i=0}^{n_3} d3_{cell} i \ldots \tag{3}$$

Taking into account Fig. 5, it can be seen that there are three clearly differentiated zones. In green, the clock periods that make the circuit work correctly. Yellow, the clock periods that cause exploitable faults to be injected for cryptanalysis. In red, the clock periods that cause the cryptocircuit to fail in many positions and its operation are incorrect and are not exploitable from the cryptanalysis point of view. Considering this figure, the proposed design methodology takes into account that the designed glitch detectors should consider, at least, from $T(min)$ to $T(max)$. The detectors designed according to the proposed methodology can detect clock pulses below $T(max)$, but such pulses are filtered out or produce non exploitable faults.

4 Clock manipulation detector design methodology

With the aim of designing a proper clock manipulation detector, we have considered both FPGA and ASIC implementations. In this sense, it is necessary to split the design into two processes; firstly, it is necessary to determine the faulty period considering the maximum operating frequency of the cryptocircuit and the technology where it is going to be implemented. Secondly, once the faulty period is known, it is necessary to calculate the delay to be applied in the detector to cover the faulty periods.

To calculate the faulty period, we have consider the Algorithm 1 for the cipher operation. The algorithm is divided into two parts; the first establishes the secret key K, the plain text P_T, and the correct clock period T, and the second allows determining the faulty period T_F to be considered in the design. Firstly, the correct cipher text CT_T is produced using the previous patterns, and secondly, the clock period T is decreased, and the cipher text is generated again $CT_{T'}$. If both ciphertexts are the same, the faulty period used T' is not the desired one. When the correct cipher text and the faulty cipher text are different $CT_T \neq CT_{T'}$, it means that the faulty period has been determined $T_F = T'$. The algorithm 1 returns a single T_F value, as this is the first period

that produces the cryptocircuit vulnerability and it is from there onward that the periods should be monitored.

Algorithm 1 Faulty period determination for a cryptocircuit

```
CRYPTO PARAMETERS
    Select randomly K/P_T ∈ HEX[0, ..., F]
    select clock period T ∈ [1, ..., 100]
    return K, P_T, T
MINIMUM PERIOD DETERMINATION(K, P_T, T)
    K_c/P_Tc ← K/P_T
    Process of CT(K_c/P_Tc) for T → CT_T
    CT_T' ← CT_T
    while CT_T = CT_T'
    T' ← T − i for i = 1, ..., T
    Process of CT(K_c/P_Tc) for T' → CT_T'
    if CT_T ≠ CT_T' then
        T_F ← T'
    end if
    return T_F
```

Once the faulty period is known, the second part of the design methodology consists of determining the necessary delays to be added to the detector. For this purpose, we can consider the Algorithm 2. This algorithm allows to determine the necessary delay D and the number of cells N_c. To this end, this process considers whether the technology is FPGA or ASIC through the W parameter. If FPGA is considered, the number of cells N_c must be an even number, and the delay D is obtained using the sum of the cells. For FPGA the type of cell is always the same, the inverters, due to the fact that in this technology it is not possible to add cell delays at a desired point. Obviously, the delay offered by the inverter is different depending on the FPGA. In the case of ASIC technology, it is possible to select different cell delays, and thus it is necessary to consider all the cell possibilities. To this end, the greatest cell delay is selected $Z(x)$ and is used to determine if the minimum faulty period is achieved, if not, the consecutive cell delay is selected, etc. After the iterative process, the algorithm returns the delay and the number of cell delays of each type that allow obtaining the mentioned delay.

According to this design methodology, it is also possible to design a group of detectors, placed in cascade, that allows a greater range of faulty frequencies to be covered. That is, it is possible to place two or more detectors, for instance, one of them for high speed short pulses and others for lower frequencies. This makes it possible to cover more possibilities of faulty frequencies and thus to have a higher fault coverage.

5 Experimental results

In order to test the detection scheme, we have designed two experimental setups. The first one is oriented to FPGAs and the second one is oriented to ASICs. Both setups use the

Algorithm 2 Calc. of delay, type and number of cells

```
D CALC(W, Z, T_F)
    if W = 0 then  − − (FPGA case)
        N_c = (T_F/2)/Z → N_c ∈ ℕ
        D = Σ_{i=0}^{N_c} d_cell i
    else  − − (ASIC case)
        C = Z(x) for x = 1, ..., n_z
        i = y for y = 1, ..., m
        n_i = 0
        while n > 1
            n = (T_F/2)/C
            n_i + +
        N_c(x) = n_i
    end if
    D = Σ_{i=0}^{N_c(1)} d1_cell i + Σ_{i=0}^{N_c(2)} d2_cell i + Σ_{i=0}^{N_c(3)} d3_cell i ...
    return D, N_c(C)
```

ChipWhisperer Husky board from NewAE together with an Agilent InfiniiVision DSO7054A oscilloscope. The ChipWhisperer Husky is a board oriented to test and attack hardware implementations of cryptographic circuits. Among other features, this board allows generating clock signals and maliciously manipulating them to introduce small clock pulses into these signals. As an example, Fig. 6 shows three captures of the clock signal generated with this board. As can be seen, the clock signal has been manipulated by introducing two short clock pulses. This test was performed using a 17 MHz clock signal and the injected pulse was approximately 3% of the size of the clock period. The difference between the different pulses is the offset applied, which causes the small pulse to be injected at a higher or lower level of the clock signal. It should be noted the accuracy when the short pulse is injected. As an example, the three possibilities can be found in Fig. 6. As can be seen in a), b), and c), the short pulses can be injected at different voltage levels of the clock signal. If a) and b) are considered, the voltage levels where the short pulses are injected do not induce any fault in the cryptocircuits due to the fact that these voltage changes are out of the range that the technology considers as a change in the voltage level of the clock. The blue line in Fig. 6 shows the mean voltage value in which the signal change is susceptible to be considered a clock change by the technology in which the circuit under attack is implemented. However, if the short pulse is injected near the blue line as depicted in c), the fault injection could be obtained. The position of the short pulse can be controlled by the offset offered by the fault attack tool, the aim being to inject the pulse near the blue line. This property is known as a capability of the attacker when the attack scenario is defined.

5.1 Simulation test

In order to test our approach, we have designed a simulation system for FPGA and it has been tested using Xilinx

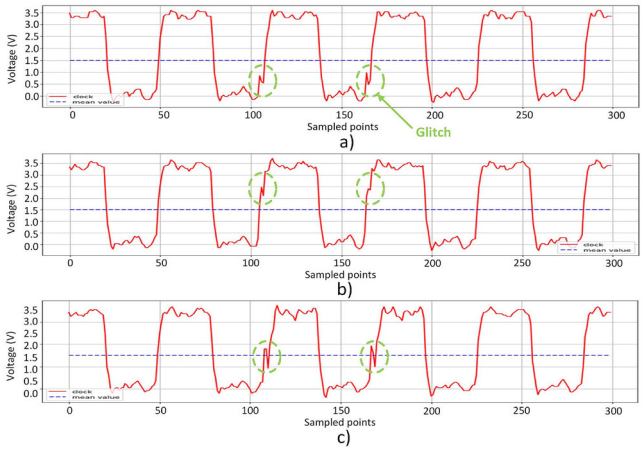


Fig. 6 Short pulse injection in the clock line. a) Short pulse below the mean voltage, b) short pulse above the mean voltage and c) short pulse just at the mean voltage. Clock frequency 17 MHz, short pulse 3% of clock period

Vivado 2022.1. For this purpose, two specific FPGA components have been used, a Digital Clock Manager (DCM) and a BUFGMUX multiplexer. The first component is an IP core to manage the FPGA clocks and to generate different clocks from the main clock signal. In this case, two different clocks have been generated with the DCM, the main clock signal, and the clock that is ten times faster than the main clock. The BUFGMUX is a special multiplexer designed to manage fast clock signal changes as a function of an input signal. Using these two components, it is possible to generate a short pulse injection in the clock signal and to test the proposed approach in simulation mode.

The results of these simulations can provide a result of the behavior of the cryptocircuit under test when a short pulse is injected into the main clock line and to test whether the clock manipulation approach detects the manipulations of the clock. The simulations used the FPGA model that has been used for the experimental tests, an Artix-7 100T with a speed grade of -2. This device has a maximum operating clock frequency limited by its internal clock buffers, approximately 628 MHz. It should be noted that in simulation, the design tool is very pessimistic, and time errors tend to appear earlier than in real implementation cases. Using the proposed simulation tests with the DCM and BUFGMUX, post-place and route simulations have been generated using different clocks. It has been found that when small clock pulses close to 615.38 MHz are injected, despite being timing simulations, errors appear and faults are injected into the circuits. Therefore, simulations could tend to inject faults with a lower frequency than real implementations, but tests could be useful to designers. With these simulation results, a designer will be able to define the frequency range for which he should design the detectors to protect the circuits using the proposed methodology.

Table 1 Report of the area in LUTs for the FPGA implementations

	AES (LUTs)	Trivium (LUTs)
Unprotected	2629	521
Protected	2636	524

5.2 Implementation and setup for FPGA

For the FPGA hardware implementation an Artix-7 XC7A100T has been used. For this implementation, the application of the delays has been done using a string of inverter components to add the delay to the clock signal. The clock signal is connected to a standard flip-flop and is sampled by the delayed clock signal. In this case, an eight-inverter string was selected as a delay to detect manipulations. Each inverter has a delay from input to output of 0.148 ns, the nets between them 0.127 ns to 0.482 ns, the input net 2.459 ns, and the output net which is connected to the flip-flop 0.681 ns, being in total 6.334 ns the delay of the clock signal. It should be noted that the use of an inverter string in FPGA is limited by the technology itself and the way it is designed, that is, the designer may use time and layout constraints to position these cells as close as possible or let the design software implement these cells without constraints and indications. This will cause the indicated delay to be higher or lower depending on the design. In our case, we have opted for routing without any type of constraint, making use of the tool automatically, with the possibility of adjusting the times if necessary. Note that the use of inverter cells forces one to use an even number of inverters since the use of an odd number of cells would lead to obtaining an inverted clock signal, and this is not the target implementation. With this configuration, the FPGA implementation has a delay of 6.334 ns on the clock line. This delay allows to know if different short pulses are injected in the main clock of the cryptocircuit. The FPGA area occupied by the respective unprotected and protected cryptographic circuits is shown in Table 1.

In Fig. 7 it can be seen the setup for testing the scheme in FPGA implementations. As can be seen, (1) the Agilent InfiniiVision DSO7054A oscilloscope was used for the trace capture with 4 G/samples and a bandwidth of 500 MHz, (2) the NewAE Husky attack tool was used for the short pulse injection, (3) the NewAE CW305 board with an Artix-7 FPGA used as a cryptocircuit target and (4) the computer for controlling the Husky board, for sending the parameters for the attacks and reading the data with Python.

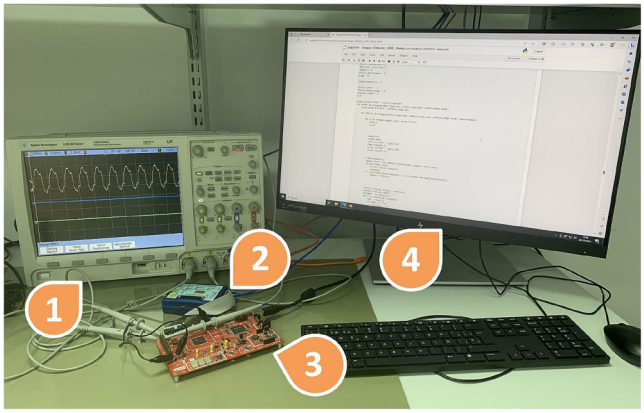


Fig. 7 Setup for the FPGA implementation

Table 2 Report of the area in μm^2 for the ASIC implementations

	AES (μm^2)	Trivium (μm^2)
Unprotected	87565.00	7331.89
Protected	87615.27	7391.16

5.3 Implementation and setup for ASIC

For the ASIC implementation a TSMC-90 nm technology was selected as a demonstration vehicle. The ASIC circuit has been designed by the authors of the paper. The AES and Trivium ciphers have been designed and fabricated in TSMC 90 nm ASIC technology with the detection scheme. The solution adopted in this technology was different from the implementation of FPGA due to the fact that inverter cells have very little delay compared to the inverters of FPGA. In this sense, in the TSMC 90 nm technology, there are different delay cells called DELX with different characteristics. In the case of TSMC 90 nm the Del4 was selected and connected in serial performing a string of three delays with 6.6 ns delay in total. In Fig. 8 it can be seen the TSMC 90nm layout of the whole design (top) and the proposed scheme using the flip-flop and DEL cells (bottom). As can be seen, the inclusion of this type of protection has a reduced impact on area consumption, less than 0,1%, due to its minimal resource requirements. The ASIC area occupied by the respective unprotected and protected cryptographic circuits is shown in Table 2.

In Fig. 9 it can be seen the setup for testing the scheme in ASIC implementations. As can be seen, (1) the Agilent InfiniiVision DSO7054A oscilloscope was used for the trace capture with 4 G/samples and a bandwidth of 500 MHz, (2) ASIC board with a TSMC 90 nm ASIC used as a cryptocircuit target, (3) NewAE Husky attack tool was used for the short pulse injection, and (4) Computer for controlling the Husky board, for sending the parameters for the attacks and reading the data with Python.

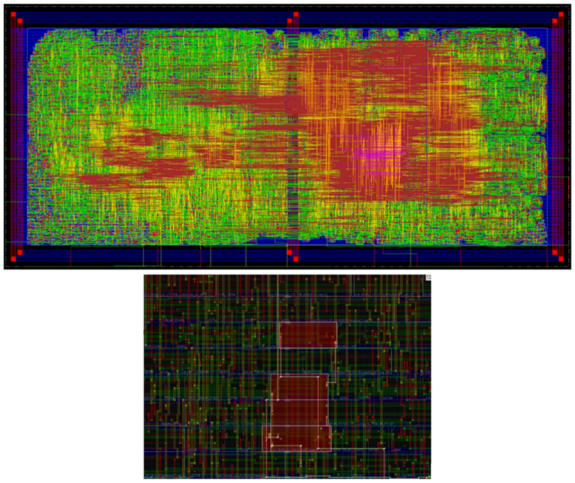


Fig. 8 Layout of the ASIC. Top: Complete layout of the circuit. Bottom: Cells of the detection scheme

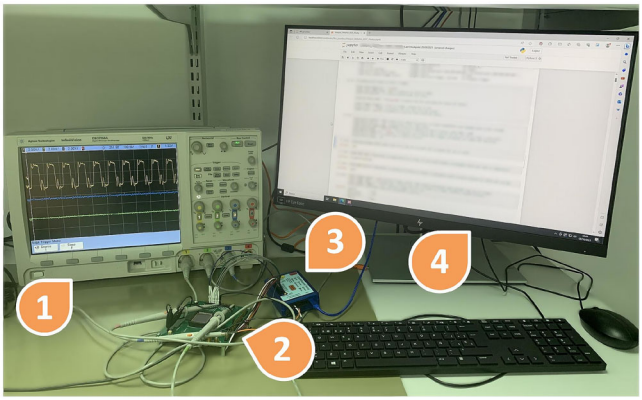


Fig. 9 Setup for the ASIC implementation

5.4 Results

To carry out the tests and evaluation, the proposed design methodology was followed. To this end, the vulnerability of both ciphers has been evaluated, i.e., the maximum frequency at which fault insertions appear has been searched, and, once found, the design of the glitch detectors has been proposed to determine whether these potentially dangerous frequencies are being injected into the system. This must be taken into account because the circuit must be protected against the potentially dangerous fault frequencies at which fault injections appear. Including frequency modification detectors that do not alter the operation of the circuit, in addition to being costly in area and time, would be irrelevant since the cryptographic circuit is not affected in its operation for frequencies that are not potentially dangerous.

However, before specifically explaining the functionality of the ChipWhisperer Husky tool for clock generation and glitch injection, it should be noted that an experimental clock frequency of 10 MHz has been used as the master clock,

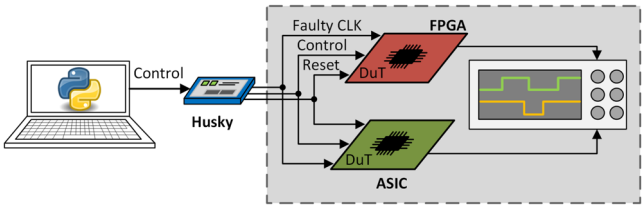


Fig. 10 Schematic representation of the setups

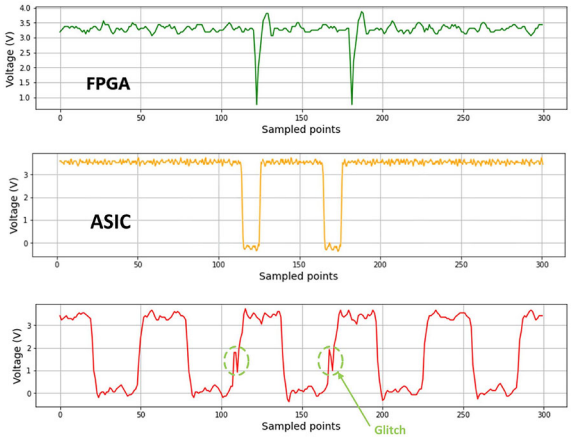


Fig. 11 Short pulse injection in the main clock signal. Green: Error signal detection; Orange: Trigger signal; Red: short pulse injection in the clock signal

which is considered optimal. Glitches with frequencies that were identified as potentially dangerous in the early stages of the design methodology were injected into this clock signal. These frequencies were deemed likely to induce faults in the ciphers used as test vehicles. It is important to clarify that using a higher or lower frequency, or performing the evaluation tests with different frequencies, is unnecessary. This is because the ChipWhisperer Husky tool for generating glitches operates (in brief) with percentages of the master frequency used to create the glitches. Specifically, starting from a master clock frequency, the tool generates a glitch at a percentage determined by the attacker and progressively decreases this percentage until the value is found that causes faults in the cipher.

The key point is that the fault mechanism is determined by an absolute fault-inducing frequency, rather than by the specific value of the master clock. In our experiments, the glitch generator operates by deriving a higher-frequency disturbance from a given master clock. Although the glitch is usually expressed as a percentage of the master clock frequency, this representation can be misleading. For example, a 5% pulse applied to a 10 MHz master clock corresponds to a 200 MHz glitch. If the master clock frequency is increased to 20 MHz, the same fault condition manifests as a 10% pulse.However, the underlying frequency inducing the fault remains unchanged at 200 MHz. This demonstrates that the malfunction of the encryption is determined by the reach of a critical frequency threshold, regardless of the master clock selected. Consequently, varying the master clock frequency provides no additional information about the fault behavior, as the system fails systematically when the same absolute fault frequency is reached. For clarity, the results of the manuscript are presented for a representative master frequency. However, the detectors have been thoroughly evaluated at multiple master frequencies, confirming that the ciphers fail at the same fault-inducing frequency, regardless of the master clock used. Furthermore, higher fault frequencies have also been explored, exceeding the minimum threshold at which the cipher begins to malfunction.

For illustrative purposes, Fig. 10 shows the setup used to test the implementations in general. The PC controls the Husky tool, which sends the patterns to either of the two technologies, and by means of the oscilloscope it is possible to observe the capture of the attacks. Considering the setups of Fig. 7 and Fig. 9, several tests have been carried out to properly characterize the detection scheme in both FPGA and ASIC. In Fig. 11 can be seen the capture of the clock manipulation detection. In green, the error or detection signal for the FPGA implementation, in orange, for the ASIC implementation, and in red, the clock signal with short pulse injections. As can be seen in this figure, two short pulses were injected into the clock signal, and, in both cases, the proposed solution is capable of detecting them, triggering the error signal from 3.3 to 0 V (logic 1 and logic 0). This error signal allows us to avoid the output of the cryptocircuit or just to notify the scenario where the circuit is under attack. It should be noted that the difference between both detection signals is due to the fact that they are different technologies, and therefore different behaviors are obtained. In the case of the ASIC, this technology is faster than the FPGA and, therefore, the time that the glitch detection is active is longer in this case.

For the test, the NewAE Husky attack tool was used. This tool is a solution for voltage and clock fault injections. In this case, only clock fault injection was used. To understand how this tool works, it is necessary to explain the variables width and offset. Both variables allow to configure the short pulse that is going to be injected in the clock line, one of them, defines the length of the pulse and the other one, define the position where the pulse is injected. Briefly, the main clock used for the cryptocircuits was 10 MHz, the offset was configured to inject pulses from 50% of the clock period, during the rising edge, and the width was configured to have 0 to 3360 steps. These steps depend on the Mixed-Mode Clock Manager (MMCM) of the internal FPGA of the tool used to generate the small pulses. The internal frequency of the attack tool was set to 600 MHz. Therefore, the higher the step value, the larger the injected pulse, which depends on

600 MHz. Note that a 10 MHz cryptocircuits master clock has been used since the objective is to be able to test the scheme and to avoid possible sampling errors due to bandwidth filtering of the oscilloscope. However, using this frequency or a higher one does not affect the final measurement results, as the detectors are designed to evaluate possible injections in the clock signal, and the delay in a high frequency clock signal will be equal to the delay in a lower frequency signal and, therefore, it will be detected in the same way. As illustrated in two paragraphs earlier.

The results of the security evaluation of the unprotected AES show that the injection of a short pulse into the main clock signal in the last rounds of the cipher makes it possible to modify the result and to achieve a successful attack. The short pulse was generated using the previous explained system and the pulse was approximately 3.9% of the clock period. In the case of the unprotected Trivium stream cipher, the results show that the cipher is vulnerable to attacks when the clock is manipulated (when a short pulse of 3.5% of the clock period is applied). For Trivium, an average of two consecutive pulses are required for the fault injection. The use of two short pulses is due to the fact that the stream cipher are based on shift register and low combinational logic if it is compared with a block cipher like AES, thus, more short pulses are necessary to inject effective faults. Note that, in both cases, these tests are close to real scenario and the attacker only has access to the control and clock signal, therefore we do not know the position, nor the length of the fault nor the operation of the cipher that was affected by the attack, we focus on the detection of malicious manipulation.

Taking into account Fig. 5, the detectors implemented following the presented design methodology have been tested using glitches from $T(min)$ onward, considering a large number of faulty periods. Specifically, more than one million tests have been performed with the periods from which cipher faults occurred until the glitch was so small that it was filtered out by the FPGA and ASIC, i.e., illustratively yellow and red zones in Fig. 5. For simplicity, the Husky tool settings are not listed, since they are Python-configured variables that are executed automatically, giving continuous values to the width and offset mentioned above.

5.5 Comparison with other schemes

In order to properly compare and evaluate the cost of the proposed solution, differential metrics were used, presenting the ratio between the unprotected and protected implementations per each cryptocircuit. The Xilinx Vivado 2022.1 and Cadence tool software has been used to obtain the values shown in Table 3. It should be noted that there are many protection schemes reported in the literature, but in this comparison, only those schemes with the best balance between area/frequency degradation and fault coverage are shown. On the other hand, the selected schemes are those that have been evaluated by their authors and show the highest fault coverage, being schemes capable of detecting single bit, multi bit, single byte and multi bytes. For further information, see references [15, 17] for comprehensive comparisons of all countermeasures added in this table.

If the AES cipher is considered, the countermeasures presented in [18] [19] are based on information redundancy and algorithm oriented. These schemes are able to protect against the types of faults enumerated before, but their protections are partial, being the first one oriented to protect the state matrix and S-box operations but not the KeySchedule, and the second one oriented to protect just the S-box. As can be seen, the level of protection is high for both, but the area overhead and frequency degradation are far from the data obtained by the proposed countermeasure. If the countermeasure presented in [16] is considered, it can be seen that the attack detection is 100%. In this case, the solution has no frequency degradation and its area overhead is close to 90% due to the fact that it is a scheme based on hardware redundancy. If a comparison is made between this scheme and the one proposed in this paper, it can be seen that both are able to detect 100% attacks, but the proposed solution in this paper has a better trade-off between area and frequency penalties. The scheme [20] is based on a combination of protection schemes, being able to offer a 100% of fault detection, nevertheless it is algorithm oriented and compared with the proposed solution, this scheme has a high penalty in area and frequency. The schemes [17].1 and [17].2 are based on information redundancy and 100% attack detection. These schemes are algorithm oriented and they are able to detect different types of faults, but the area overhead is at least 18% greater than our proposal.

If it is considered the [25], only synthesis ASIC data are provided, but the authors implement the countermeasure also in FPGA. As well as our proposal, this scheme has a protection level that covers the algorithm and the system. This scheme has a minimal area penalty, being 0.18% more in the case of ASIC technology and 0.03% less in the case of FPGA technology. This scheme, in contrast to our scheme, needs to implement ring oscillators, counters, and non-volatile memory to compare the possible injection of short pulses in the clock while the presented scheme detects the short pulse at the time of injection and does not need any memory or counter which are susceptible to not working properly when the system is under attack in extreme conditions. In the case of [26] the system is applied to an AES but the authors do not make the area and frequency penalty analysis, therefore this scheme is not included in the table. However, in addition to the combinational logic, they use BRAMs, which would imply a higher area cost than the one presented in this work. When comparing with the scheme in [27], it can be observed that, although the protections are not applied to the same cipher, being implemented on DES in this case,the abso-

Table 3 Comparison with different detection schemes

Scheme for AES	Protection level	Area overhead	Frequency degradation	Attack detection	Technology
Unprotected	–	0%	0%	0%	Any
[18]		77%	24%	90%	Virtex-E
[19]		25%	12%	98%	Virtex-5
[16]		87%	0%	100%	Virtex-5
[20]	Algorithm	38%	22%	100%	Virtex-6
[17].1		18%	0%	100%	Artix-7
[17].2		42%	0%	100%	Artix-7
[25]	Algorithm + System	0.23%	0%	No Info.	IBM-90nm
[27]	Algorithm DES + System	1,59 - 5,84%	No Info.	100%	Virtex-7
[28]	Algorithm	10,4%	No Info.	No Info.	TSMC-65nm
Our proposal	Algorithm	0.26%	0%	100%	Artix-7
	+ System	0.05%	0%	100%	TSMC-90nm
Scheme for trivium	Protection level	Area overhead	Frequency degradation	Attack detection	Technology
Unprotected	–	0%	0%	0%	Any
[15].1		100%	2%	98.36%	Artix-7
[15].2	Algorithm	34%	33%	87.85%	Artix-7
[15].3		6%	17%	79.48%	Artix-7
[15].4		38%	36%	99.99%	Artix-7
Our proposal	Algorithm	0.56%	0%	100%	Artix-7
	+ System	0.81%	0%	100%	TSMC-90nm

lute area overhead between the unprotected and protected designs ranges from 1.59% to 5.84%. In the FPGA case, this represents at least an additional 1.33% overhead compared to our proposed approach. This increase is expected, considering that the implementation requires additional hardware resources, including two counters, comparators, and the corresponding delay configurations. In this case, no information is provided regarding frequency degradation. However, it is specified that, for each target operating frequency, the protection scheme must be redesigned, as it depends on the timing analysis of the specific circuit at that particular frequency. In the case of [28], the proposed scheme presents a higher area overhead, reaching 10.4%, while no information is provided regarding frequency degradation or fault detection performance. Instead, the authors analyze the relationship between the enable signal activation time and voltage degradation. For ASIC implementations, the system is not experimentally validated and only static analyses are reported. In the FPGA case, the test is conducted on a SASEBO-G platform. However, detailed FPGA resource utilization data are not provided. Considering the reported ASIC area overhead, this scheme introduces an additional 10.35% compared to the proposed approach. Nevertheless, the authors claim that the countermeasure is applicable to the detection of Differential Fault Analysis (DFA) and Fault Sensitivity Analysis (FSA) attacks.

If the Trivium cipher is taken into account, the schemes [15].1, [15].2, [15].3 and [15].4 must be considered. The scheme [15].1 is based on hardware redundancy, specifically whole cipher redundancy, which presents a huge penalty in the area. On the other hand, [15].2 is an information redundancy in which a signature is added to the inner state of the cipher. This scheme presents a 34% of area penalty and a 33% frequency degradation with respect to the solution presented. The scheme [15].3 is based on temporal redundancy, where the scheme repeats the feedback operations of the cipher. This scheme is closest to our proposal with a 6% area overhead, but, in contrast, it has a higher frequency degradation and the attack detection is about 21% lower. Finally, the scheme [15].4 is based on the combination of [15].2 and [15].3 and provides better fault detection than the previous ones. However, the area cost reaches 38% and the frequency degradation is very high, reaching up to 36%. Taking into account these results, the proposed solution has the lowest impact on the area consumption. This is essential due to the fact that Trivium is a lightweight cipher oriented to applications where the area consumption is a concern.

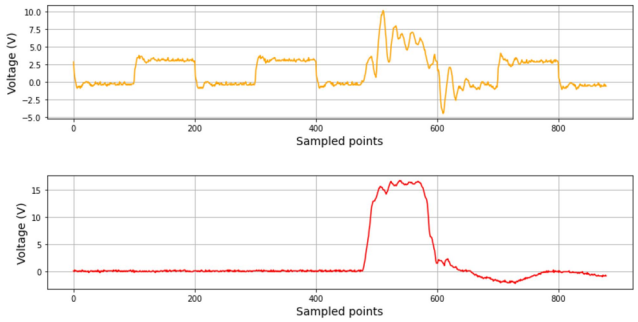


Fig. 12 Effects over the clock signal when the EMP is injected. Orange: clock signal; Red: EMP injection

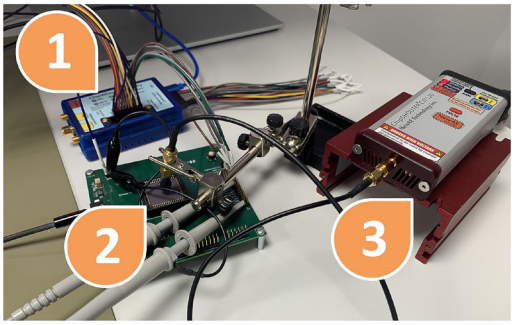


Fig. 13 Setup of the EMP fault attack. (1) control board Husky tool, (2) the injection tip and the board where the integrated circuit is located and (3) the EMP tool ChipShouter

5.6 Evaluation of designs against Electromagnetic attacks

Electromagnetic fault attacks are based on the introduction of errors in an integrated circuit using an electromagnetic pulse (EMP). When the electromagnetic field of the EMP penetrates the device, it produces anomalous voltage differences and currents within the circuit components. Inducing Foucault currents on the chip surface can cause a fault of up to a single bit [32]. In addition to the security aspect, there is also the characterization and study of the resilience of integrated circuits to fault insertions produced by the environment itself, such as electromagnetic interference or cosmic rays, when circuits are used in space applications [8, 33].

If the clock signal is considered, a representative effect on the clock signal when an EMP is induced can be observed in Fig. 12. As can be seen, this type of attacks affect the voltage levels of the clock signal, and on the other hand, these disturbances can be interpreted by the circuit as short pulses and therefore as fault injections. These disturbances are susceptible to be detected if a countermeasure protecting the clock signal is designed and implemented in the circuits.

Considering that, the countermeasure methodology proposed in this paper has been tested against EM fault injections with the aim to characterize the possibility of using it as a detection against this type of attack. For this purpose, the same setups for FPGA and ASIC technology were used, but in this case the ChipShouter tool from NewAE was added. This tool is an electromagnetic pulse generator that allows EM attacks on the devices. Note that the experimental tests were performed on both FPGA and ASIC. The setup used to test the ASIC technology is shown in Fig. 13. For this purpose, different voltage ranges allowed by the tool were used. The 4 mm injection tip that generates clockwise EM fields was used in the tests. Figure 13 shows (1) Husky tool control board, (2) injection tip and the board where the integrated circuit is located, and (3) the ChipShouter EMP tool.

In Fig. 14 the response of the detection scheme against EMP is shown. This figure shows the instant of time at which

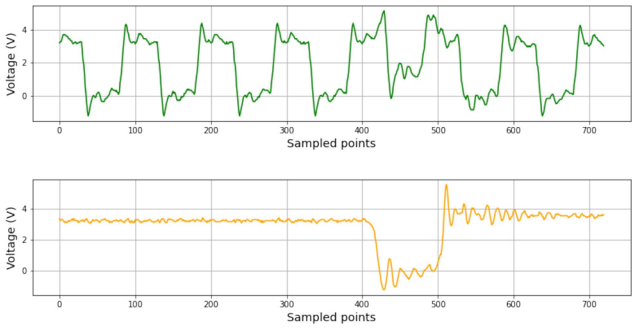


Fig. 14 Detection of EMP triggering the error signal. Green: clock signal; Orange: error signal

the EMP was injected. As a result of the injection of EMP, the clock signal has a disturbance or noise in the form of short pulses that are detected by the proposed scheme. On the other hand, as can be seen, this detection is greater than those produced when the attack is performed only in the clock line through the short pulse injections. The results show that using voltage ranges between 100V (minimum) to 500V (maximum) to generate EM pulse injections, the proposed scheme is able to detect all the EM fault attacks in both platforms. To evaluate the designs, more than 100K tests have been performed. This number is lower than those performed for clock manipulation attacks and is due to the fact that this type of attack is more aggressive, more damaging against the devices, and its cost in time is much higher.

Therefore, the use of this scheme can add extra protection due to the fact that it allows us to detect if the circuit is under an EM fault injection attack and, at the same time, to detect clock manipulation attacks. Furthermore, it should be noted that this countermeasure is applicable not only to the clock signal but is applicable to any signal susceptible to being vulnerable to introduce faults when it changes rapidly through small pulses, such as circuit control signals.

6 Conclusions

This paper presents the development of a hardware-oriented protection methodology to strengthen encryption algorithms against fault injection. The presented scheme is capable of detecting attacks on the clock signal and has been experimentally tested using FPGA and ASIC technologies when implemented in parallel with standard ciphers such as AES and Trivium. The results show that the presented scheme is capable of detecting 100% of the attacks performed (more than 1 million) using different fault frequencies and whose impact on area usage is negligible and null in terms of frequency penalty. Furthermore, it has been experimentally tested as a countermeasure against electromagnetic attacks and the results have shown that in 100% of the tests (more than 100K tests), the scheme is able to detect that the circuit is being attacked with EM pulses. The selection of the number of tests was determined by the time required to carry out a complete attack. The presented scheme has been compared with previous schemes reported in the literature, both with the same topology and with other protection schemes.The results show that the proposed scheme has the best balance between resource consumption and protection. The results have shown that the use of this scheme is applicable to protect both FPGA and ASIC implementations where resource constraints are very high such as in IoT applications. Finally, the results show that this type of scheme is a combined countermeasure capable of detecting clock signal fault injection attacks and also EM attacks, thus forming a combined countermeasure whose consumption is practically negligible. Future work will focus on optimizing and developing the algorithms presented with the aim of improving the schemes designed, which will be implemented in the future to strengthen the protection of vulnerable systems. Additionally, we intend to study the proposed system in different technologies and extend its use against laser fault injections.

Acknowledgements Authors want to thank the project GreenCrypt, with grant PID2024–158044OB-I00, funded by MI-CIU/AEI/10.13039/501100011033, and FEDER, European Union. Authors want to thank the Ministry for Digital Transformation and Public Function through grant USECHIP (TSI-069100–2023-001) of PERTE Chip Chair program, funded by European Union–Next Generation EU.

Author contributions Conceptualization, F.E.P.-O., E.T.-S. and A.J.A.-J.; methodology, F.E.P.-O., A.J.A.-J., J.M.M.-G. and E.T.-S.; software, F.E.P.-O. and J.M.M.-G.; validation, F.E.P.-O., J.M.M.-G., and E.T.-S.; formal analysis, F.E.P.-O., J.M.M.-G., and E.T.-S.; investigation, F.E.P.-O., J.M.M.-G., A.J.A.-J. and E.T.-S.; resources, E.T.-S.; data curation, F.E.P.-O.; writing—original draft preparation, F.E.P.-O., and E.T.-S.; writing—review and editing, F.E.P.-O., J.M.M.-G., A.J.A.-J. and E.T.-S.; visualization, F.E.P.-O., and E.T.-S.; supervision, A.J.A.-J.; project administration, A.J.A.-J.

Funding Funding for open access publishing: Universidad de Sevilla/CBUA

Data availability No datasets were generated or analysed during the current study.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Krishnan, A., et al.: "State of IoT Summer 2024," in *IoT Analytics* (2024)
2. Dutta, I. K., Ghosh, B., Bayoumi, M.: Lightweight Cryptography for Internet of Insecure Things: A Survey, In: *IEEE 9th Annual Computing and Communication Workshop and Conference (CCWC'19)*, pp. 475–481, (2019)
3. Gunathilake, N. A., Al-Dubai, A., Buchana, W. J. Recent Advances and Trends in Lightweight Cryptography for IoT Security. In: *16th International Conference on Network and Service Management (CNSM'20)*, pp. 1–5, (2020)
4. Dhanda, S.S., Singh, B., Jindal, P.: Lightweight cryptography: A solution to secure IoT. Wireless Pers. Commun. **112**(3), 1947–1980 (2020)
5. Kazemi, Z., Fazeli, M., Hely, D., Beroulle, V.: Hardware Security Vulnerability Assessment to Identify the Potential Risks in a Critical Embedded Application. In: *IEEE 26th International Symposium on On-Line Testing and Robust System Design (IOLTS'20)*, pp. 1–6, (2020)
6. Kazemi, Z., Hely, D., Fazeli, M., Beroulle, V.: A review on evaluation and configuration of fault injection attack instruments to design attack resistant mcu-based IoT applications. Electronics **9**(1153), 1–27 (2020)
7. Kaur, S., Singh, B., Kaur, H.: Stratification of hardware attacks: side channel attacks and fault injection techniques. SN Comput. Sci. **2**(183), 1–13 (2021)
8. Bar-El, H., Choukri, H., Naccache, D., Tunstall, M., Whelan, C.: The sorcerer's apprentice guide to fault attacks. Proc. IEEE **94**(2), 370–382 (2006)
9. Biham, E., Shamir, A.: Differential fault analysis of secret key cryptosystems. In: Advances in Cryptology (CRYPTO'97): 17th Annual International Cryptology Conference Santa Barbara, California, USA August 17–21, Proceedings 17, pp. 513-525, (1997)
10. Potestad-Ordóñez, F.E., Jiménez-Fernández, C.J., Valencia-Barrero, M.: Vulnerability analysis of trivium implementations. IEEE Trans. Very Large Scale Integ. (VLSI) Syst. **12**(25), 3380–3389 (2017)
11. Mohamed, M. S. E., Buchmann, J.: Mutant differential fault analysis of Trivium MDFA. In: Information Security and Cryptology-ICISC 2014: 17th International Conference, Seoul, South Korea, December 3-5, pp. 433-446, (2015)
12. Gruber, M., Probst, M., Tempelmeier, M.: Statistical ineffective fault analysis of GIMLI. In: 2020 IEEE International Symposium on Hardware Oriented Security and Trust (HOST), pp. 252-261, (2020)

13. Nakamura, K., Hori, K., Hirose, S.: Algebraic fault analysis of SHA-256 compression function and its application. Information **12**(10), 433 (2021)
14. NIST National Institute of Standards, Cryptography. In: *https://www.nist.gov/cryptography*, 2024
15. Potestad-Ordóñez, F.E., Tena-Sánchez, E., Mora-Gutiérrez, J.M., Valencia-Barrero, M., Jiménez-Fernández, C.J.: Trivium Stream Cipher Countermeasures Against Fault Injection Attacks and DFA. IEEE Access **9**, 168444–168454 (2021)
16. Joye, M., Manet, P., Rigaud, J.B.: Strengthening hardware AES implementations against fault attacks. IET Inform. Secur. **1**(3), 106–110 (2007)
17. Potestad-Ordóñez, F.E., Tena-Sánchez, E., Acosta-Jiménez, A.J., Jiménez-Fernández, C.J., Chaves, R.: Design and Evaluation of Countermeasures Against Fault Injection Attacks and Power Side-Channel Leakage Exploration for AES Block Cipher. IEEE Access **10**, 65548–65561 (2022)
18. Karpovsky, M., Kulikowski, K. J., Taubin, A., Member, S.: Robust Protection Against Fault-Injection Attacks on Smart Cards Implementing the Advanced Encryption Standard. In International Conference on Dependable Systems and Networks (DSN'04), pp. 93 – 101 (2004)
19. Mestiri, H., Benhadjyoussef, N., Machhout, M., Tourki, R.: High performance and reliable fault detection scheme for the advanced encryption standard. Int. Rev. Comput. Softw. IRECOS13 **8**(3), 730–746 (2013)
20. Karri, R., Wu, K., Mishra, P., Kim, Y.: Concurrent error detection schemes for fault-based side-channel cryptanalysis of symmetric block ciphers. IEEE Trans. Comput. Aided Des. Integr. Circuits Syst. **21**(12), 1509–1517 (2002)
21. Tewary, A., Mandal, S., Chakrabarti, A., Saha, D., Adhikari, A.: Differential Fault Analysis of Trivium Using Artificial Neural Network on SoC Platform. In Emerging Electronic Devices, Circuits and Systems: Select Proceedings of EEDCS Workshop Held in Conjunction with ISDCS 2022, pp. 269-280, Singapore: Springer Nature Singapore, (2023)
22. Karmakar, B., Saha, D.: DESIV: Differential fault analysis of SIV-Rijndael256 with a single fault. In: 2020 IEEE International Symposium on Hardware Oriented Security and Trust (HOST), pp. 241-25 1, IEEE, (2020)
23. Ghosal, A.K., Sardar, A., Chowdhury, D.R.: Differential fault analysis attack-tolerant hardware implementation of AES. J. Supercomput. **80**(4), 4648–4681 (2024)
24. Wan, X., Zhang, J., Wu, W., Cheng, S., Wang, J.: Differential Fault Analysis Against AES Based on a Hybrid Fault Model. In: International Conference on Neural Information Processing, pp. 161-171, Singapore: Springer Nature Singapore, (2023)
25. Deshpande, C., Yuce, B., Ghalaty, N. F., Ganta, D., Schaumont, P., Nazhandali, L.: A configurable and lightweight timing monitor for fault attack detection. In: *2016 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pp. 461–466, IEEE, (2016)
26. Velegalati, R., Shah, K., Kaps, J. P.: Glitch detection in hardware implementations on FPGAs using delay based sampling techniques. In *2013 Euromicro Conference on Digital System Design*, pp. 947–954, IEEE, (2013)
27. Zhang, M., Liu, Q.: A digital and lightweight delay-based detector against fault injection attacks. *2021 IEEE International Symposium on Circuits and Systems (ISCAS'21)*, pp. 1-5, IEEE, (2021)
28. Endo, S., Li, Y., Homma, N., Sakiyama, K., Ohta, K., Aoki, T.: An efficient countermeasure against fault sensitivity analysis using configurable delay blocks. *2012 Workshop on Fault Diagnosis and Tolerance in Cryptography*, pp. 95-102, IEEE, (2012)
29. Pub, N.I.S.T.F.I.P.S.: 197: Advanced encryption standard (AES). Feder. Infor. Proc. Stand. Public. **197**(441), 0311 (2001)
30. De Cannière, C.: Trivium: A stream cipher construction inspired by block cipher design principles. In: *International Conference on Information Security (ISC'06)*, pp. 171–186, (2006)
31. International Organization for Standardization: ISO/IEC 29192-3:2018. *Information Security—Lightweight Cryptography—Part 3: Stream Ciphers*; Int. Organization for Standardization: Geneva, Switzerland, (2018)
32. Quisquater, J. J., Samyde, D.: Eddy current for magnetic analysis with active sensor. In: Proceedings of eSMART, 185–194 (2002)
33. Baumann, R.: Radiation-induced soft errors in advanced semiconductor technologies'. IEEE Trans. Device Mater. Reliab. **5**(3), 305–316 (2005)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.